

Phishing Website Detection A Critical Reproduction Study

Data Science in Cyber · Final Project · Phishing Detection

Source evaluated: *shreyagopal — Phishing Website Detection by Machine Learning Techniques*

Executive Summary

This project critically evaluates a widely used phishing-detection tutorial that extracts 17 URL features, assembles a balanced dataset of 10,000 URLs (50% phishing from PhishTank, 50% legitimate), trains six machine-learning classifiers, and reports test accuracy in the mid-80% range, concluding that XGBoost (~86%) is an effective phishing detector. We reproduced the full pipeline and tested whether that conclusion is supported by the evidence.

The headline number reproduces, but it does not survive scrutiny. Our best model (Random Forest) reaches 86.9% accuracy, matching the author, but we show this figure is (i) **capped by the data, not the model**: the binarised features collapse the 10,000 URLs into only 771 distinct vectors, 63.8% of rows are ambiguous (identical features, both labels), and the Bayes-optimal accuracy ceiling is 86.8%, essentially where every model lands; (ii) **inflated by an unrealistic 50/50 class balance**: re-projected onto a realistic ~1% phishing prevalence, the precision a user would actually experience falls to 11.1%; and (iii) **partly built on a biased feature**: URL_Length equals 1 for 100.0% of legitimate rows, encoding which corpus a URL came from rather than whether it is phishing.

We further find the feature-extraction code is **not reproducible today** (it depends on Alexa rank, discontinued in 2022, and live WHOIS lookups), and that removing those features halves model quality (MCC 0.7436 to 0.433). Our verdict: the tutorial is a sound teaching example and the XGBoost-is-best claim holds, but the reported performance overstates real-world capability and should not be cited as evidence of ~86% deployable accuracy without the corrections we recommend.

1. Summary of the Source

Problem and why it matters

Phishing websites impersonate trusted brands to steal credentials and payment data. They are short-lived, high-volume, and a leading initial-access vector for breaches. Detecting them automatically from the URL or page, before a user submits data, is therefore a core defensive capability, and one well suited to machine learning because individual phishing pages share recurring structural tells.

Proposed solution, dataset and methodology

The source proposes a supervised URL classifier. It collects 5,000 phishing URLs from PhishTank and 5,000 legitimate URLs from a benign-URL corpus, then runs a custom feature extractor producing 17 features in three families: *address-bar / lexical* (presence of an IP address, '@', URL length and depth, redirection, an 'https' token in the domain, URL shorteners, prefix/suffix hyphenation), *domain / host* (DNS record existence, web-traffic rank, domain age and remaining registration period), and *HTML / JavaScript* behaviour (iFrame redirection, status-bar manipulation, right-click disabling, forwarding). All features are reduced to binary indicators except URL depth. Six classifiers are trained and compared by test accuracy, with XGBoost reported best at roughly 86%.

2. Critical Evaluation of the Author's Claims

We organise the critique around the author's explicit and implicit claims, confronting each with evidence from our reproduction. The summary verdict:

Author's claim	Our evidence	Verdict
~86% accuracy means an effective detector	Reproduced (86.9%), but it equals the 86.8% information ceiling and collapses under realistic prevalence	Partially supported
The features capture phishing behaviour	The top feature URL_Length = 1 for 100.0% of legitimate rows, so it encodes the data source, not phishing	Not supported (leakage)
The pipeline is reproducible	The extractor needs Alexa rank (discontinued 2022) and live WHOIS lookups; missing values were silently set to 0	Not reproducible
A balanced dataset is acceptable	Real prevalence is ~1%, where precision falls to 11.1%; the author never resamples or notes it	Not addressed
XGBoost is the best model	Confirmed best by both accuracy and MCC in our reproduction	Supported

2.1 Accuracy is the wrong headline metric

The dataset is a perfect 50/50 split, so a coin flip already scores 50% and accuracy is flattering by construction. More importantly, the 17 coarse features map all 10,000 URLs onto just 771 distinct vectors; 63.8% of rows are ambiguous, meaning the same feature vector occurs as both phishing and legitimate. The majority-vote (Bayes-optimal) accuracy on this representation is therefore 86.8%, and our best model (86.9%) already sits on that wall. The six models clustering in the mid-80s is not evidence that the signal is strong; they are all hitting the **same information ceiling**. Figure 1 makes this concrete.

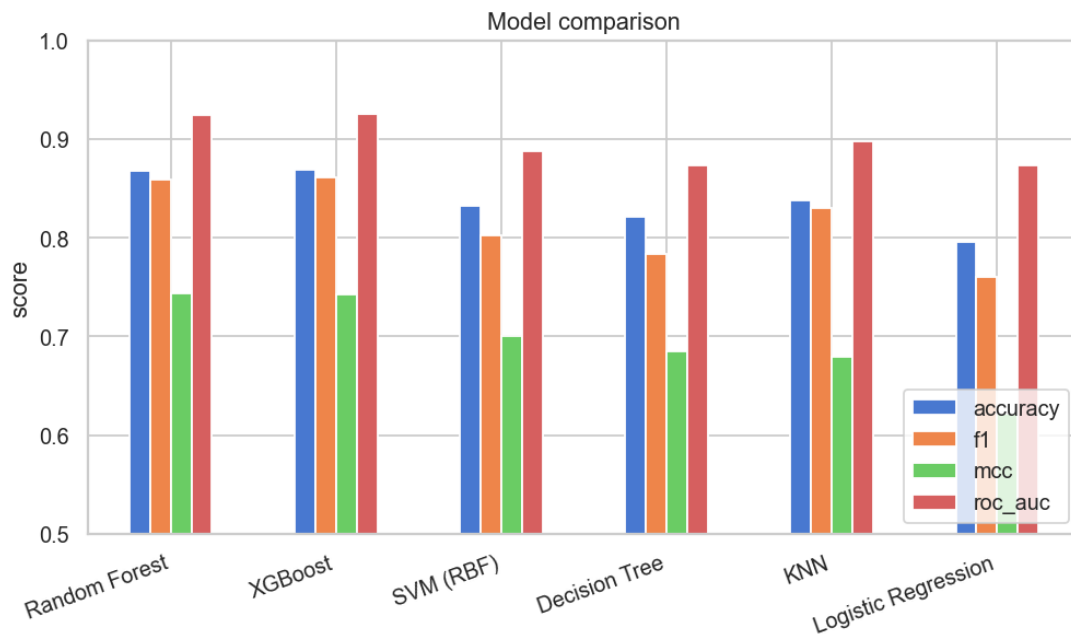


Figure 1. Model comparison. Every classifier plateaus just below the 86.8% information ceiling of the binarised features, regardless of its sophistication.

2.2 A leaky, source-revealing feature

URL_Length is the single most predictive feature, yet it equals 1 ('long URL') for 100.0% of legitimate rows versus only ~55% of phishing rows. This is not a property of phishing; it is an artefact of how the two corpora were sampled (legitimate URLs were full deep links, all long, while PhishTank entries are often bare domains). The classifier partly learns which file a row came from, a classic selection-bias leak. Figure 2 shows that, beyond URL_Length and prefix/suffix hyphenation, most flags sit close to the 0.5 base rate and carry little signal.

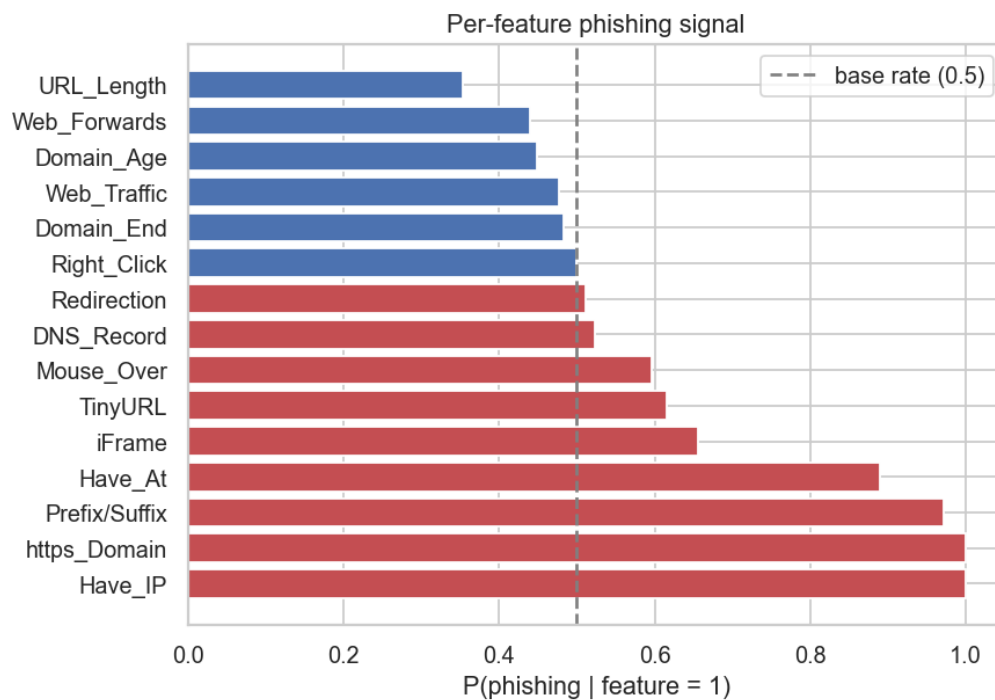


Figure 2. Probability of phishing given each binary feature equals 1. Bars near the dashed 0.5 line carry almost no discriminative information; only a few features are genuinely predictive.

2.3 The balanced-data illusion

Phishing is rare in real traffic (well under 1%). The dataset's perfect balance (Figure 3) is therefore an artefact of construction, not a reflection of deployment.

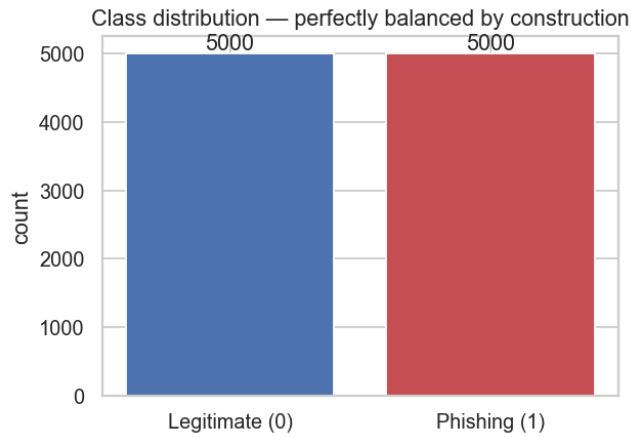


Figure 3. The dataset is split exactly 50/50 by construction, which inflates accuracy and hides the precision problem that appears at realistic prevalence.

Holding the model's measured detection rate (80.2%) and false-positive rate (6.5%) fixed and re-projecting onto lower base rates (Figure 4), the precision a user would experience drops to 11.1% at ~1% prevalence: most alerts would be false alarms. The author neither resamples toward realistic prevalence nor reports prevalence-robust metrics, so the stated performance does not represent deployment.

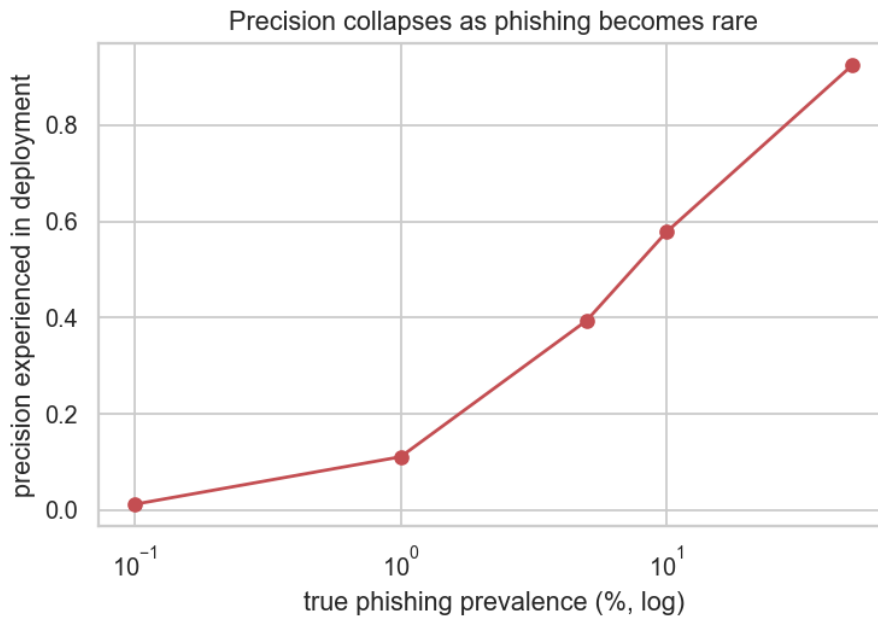


Figure 4. Precision experienced in deployment as a function of true phishing prevalence (log scale). It collapses from the dataset's optimistic value to about 11.1% at a realistic 1% base rate.

2.4 Methodology gaps

- No cross-validation in the original (a single split); we add 5-fold stratified CV.
- No discussion of the false-positive / false-negative trade-off, despite the strong cost asymmetry in phishing (a miss hands a user to an attacker).
- A purely random split over a dataset where identical feature vectors recur thousands of times causes mild train/test contamination, flattering the hold-out score.
- No temporal validation, although phishing is highly non-stationary (concept drift).

Are the conclusions justified? Partly. 'XGBoost performs best here' is justified. 'This is an effective phishing detector at ~86%' is **not** justified as a real-world claim: the number is a ceiling artefact, measured under unrealistic balance, and propped up by a biased and non-reproducible feature.

3. Feature Engineering Analysis

Was feature engineering performed? Yes, but it is largely destructive binarisation. Continuous signals (URL length, domain age, traffic rank) are thresholded into single 0/1 flags. Mathematically this discards the information needed to separate the 63.8% of rows that become ambiguous; in security terms it throws away the granularity (how long, how new) that distinguishes a borderline phishing domain from a benign one.

Transformations, scaling and encoding

- **Encoding:** none required for modelling; the features arrive pre-encoded as 0/1 indicators. We keep URL_Depth as an ordinal count rather than one-hot, preserving order.
- **Scaling:** tree models are scale-invariant; KNN, SVM and Logistic Regression are wrapped in a StandardScaler pipeline (fit on training folds only, so no leakage).
- **Feature creation:** we add a suspicion_score, the sum of the source-independent lexical flags, giving an interpretable, drift-resistant signal that isolates performance not dependent on the leaky WHOIS/Alexa features.
- **Dimensionality reduction:** PCA needs almost all components to reach 90% variance (Figure 5), so it buys no compression and harms interpretability; we reject it and use mutual-information feature selection instead.

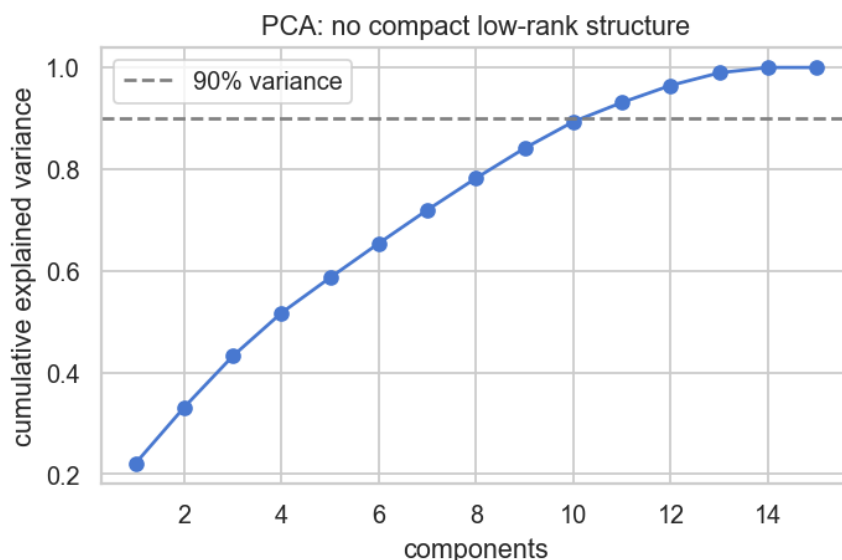


Figure 5. PCA cumulative explained variance. The binary indicators share no latent structure, so nearly all components are needed for 90% variance and dimensionality reduction adds no value.

Redundancy

Redundancy is severe: 9,229 of 10,000 rows duplicate another row in feature space, two features (https_Domain, Right_Click) are near-constant and carry no information, and several flags have near-zero mutual information with the label. The correlation structure in Figure 6 confirms it. We spot redundancy with three complementary tools (a Spearman correlation matrix, mutual-information ranking, and near-zero-variance detection) and

tackle it by dropping the constant features.

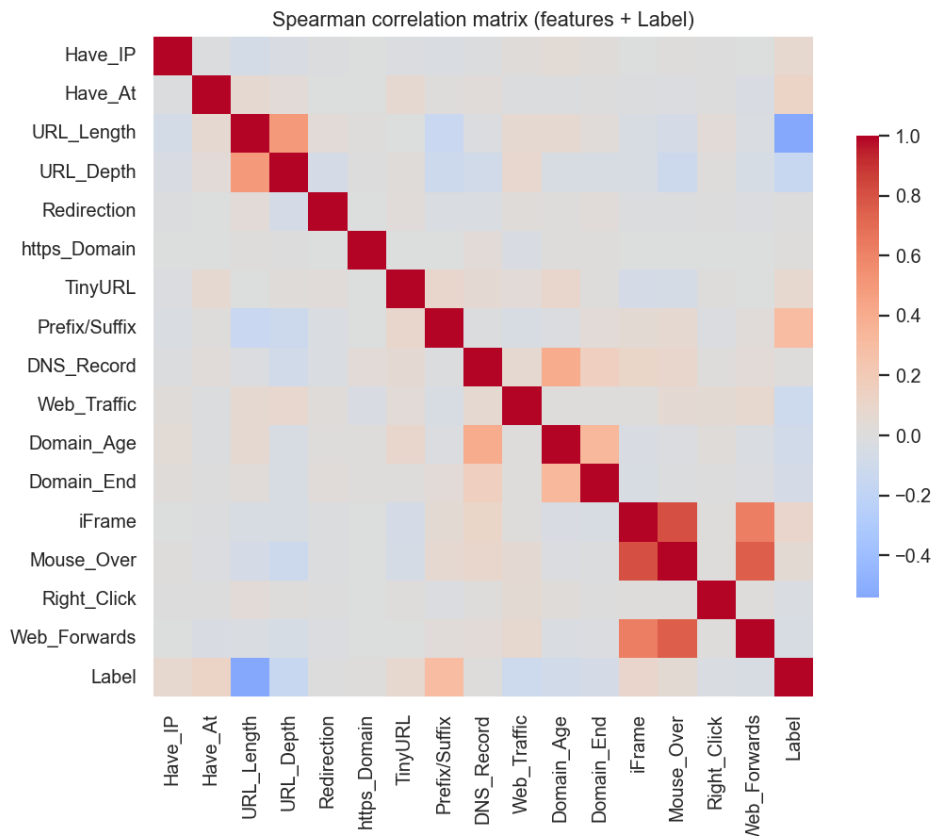


Figure 6. Spearman correlation matrix (chosen over Pearson because the features are ordinal and binary). Most off-diagonal correlations are weak, and only a few features correlate with the label.

Additional features that could help

- Character-level URL entropy and token statistics (random-looking hostnames).
- Domain and brand edit-distance and homograph (look-alike) detection.
- TLS certificate age, issuer and validity; favicon / domain mismatch.
- Hosting / ASN reputation and passive-DNS age, via stable APIs rather than discontinued Alexa rank.

4. Reproducibility Analysis

- **Can the modelling code run?** Yes; given the supplied CSV, the training and evaluation pipeline executes cleanly (we reproduced it end-to-end).
- **Can the feature extraction run?** No, not as published. It calls Alexa for web-traffic rank (discontinued in 2022) and performs live WHOIS lookups for domain age, which are slow, rate-limited and time-dependent. Re-extracting today yields different values.
- **Hidden preprocessing:** the dataset reports zero missing values, which is implausible for WHOIS-derived fields; lookup failures were most likely silently coerced to 0, conflating 'unknown' with 'absent', an undocumented and label-correlated step.
- **Files and dependencies:** the CSV and notebooks are provided, but versions are not pinned and the extractor's external dependency (Alexa) no longer exists.

Overall: the results are re-runnable on the frozen CSV but the study is not reproducible from raw URLs, a critical distinction for a security tool that must be retrained as phishing

evolves.

5. Experimental Results

We trained six classifiers with a fixed seed (42), a stratified 80/20 split, and 5-fold stratified cross-validation. Models requiring scaling were pipelined with StandardScaler. We led with MCC and F2 (beta = 2, because a missed phishing page is costlier than a false alarm) rather than accuracy. Results on the held-out test set are shown below, sorted by MCC, the prevalence-robust score.

Model	ACCURACY	PRECISION	RECALL	F1	F2	MCC	ROC_AUC
Random Forest	0.869	0.925	0.802	0.859	0.824	0.744	0.925
XGBoost	0.869	0.917	0.811	0.861	0.830	0.743	0.926
SVM (RBF)	0.833	0.983	0.678	0.802	0.723	0.701	0.888
Decision Tree	0.821	0.992	0.647	0.783	0.695	0.685	0.874
KNN	0.838	0.873	0.791	0.830	0.806	0.679	0.898
Logistic Regression	0.796	0.923	0.647	0.761	0.688	0.621	0.874

Table 1. Test-set performance, sorted by MCC.

The ranking is consistent across metrics: the tree ensembles (Random Forest, XGBoost) lead, and the linear model trails. The separability curves in Figure 7 summarise the best model's ranking quality.

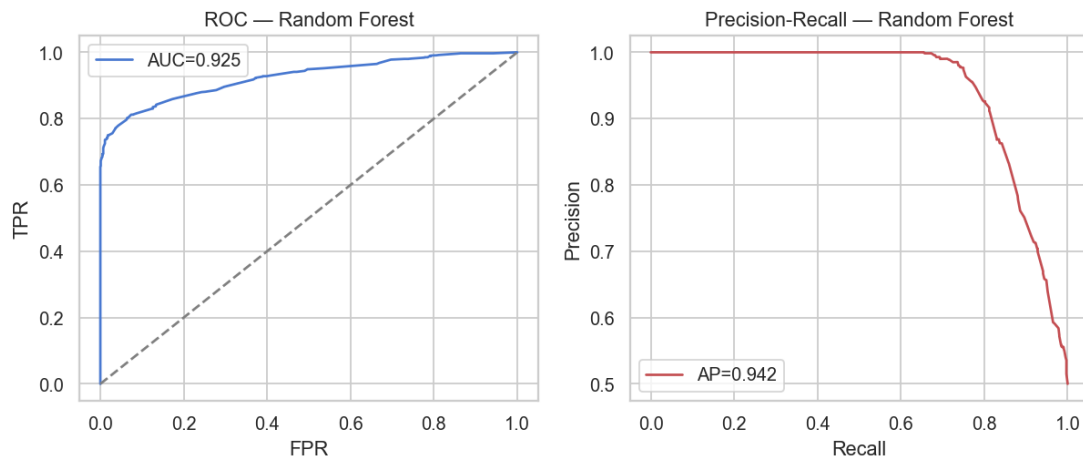


Figure 7. ROC (left) and Precision-Recall (right) curves for the best model. Both areas are high on the balanced test set, which is precisely why the prevalence correction in Section 2.3 matters.

Modifications we introduced

- Added 5-fold stratified cross-validation, MCC, F-beta, ROC-AUC and PR-AUC (the original reports accuracy only).
- Derived the Bayes-optimal accuracy ceiling from feature-space ambiguity.
- Added a prevalence-correction analysis exposing deployment precision.
- Trained a source-independent (lexical-only) model: accuracy falls to 71.5% and MCC to 0.433, quantifying how much performance leans on the non-reproducible WHOIS/Alexa features.

6. Error Analysis

At the default threshold the best model detects 80.2% of phishing (false-negative rate 19.8%) while wrongly flagging 6.5% of legitimate sites (Figure 8).

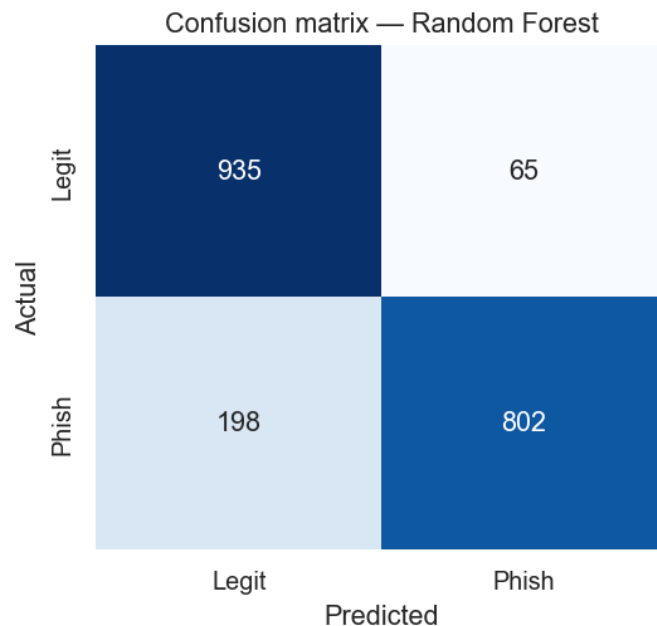


Figure 8. Confusion matrix for the best model (Random Forest). The off-diagonal cells are the errors analysed below: false negatives (missed phishing) and false positives (blocked legitimate sites).

- **False negatives** are lexically clean phishing URLs: short, no IP, no '@', no shortener. This matches modern phishing hosted on compromised legitimate domains or cloud platforms, so the model's blind spot aligns with the most dangerous real-world attacks. Each false negative delivers a user to an attacker.
- **False positives** are legitimate sites that trip a lexical flag (long or hyphenated URLs). Each erodes trust and, at scale, pushes users to disable the detector.

False-positive / false-negative trade-off. Because a miss is costlier than a false alarm, the right operating point favours recall, hence our emphasis on F2 and MCC. A deployment would lower the decision threshold and pair the model with allow-lists and a soft 'warn' rather than a hard 'block' to keep precision tolerable. The source leaves the threshold at 0.5 and never engages this trade-off.

7. Conclusions

Key findings and lessons

- The ~86% accuracy is reproducible but is an information ceiling of the binarised features, not a measure of model skill.
- On imbalanced security problems accuracy is misleading; MCC, F-beta and prevalence-corrected precision are essential, and tell a soberer story here.
- Always audit whether a 'predictive' feature is really measuring the label-generation process; URL_Length here measures data provenance.
- A security model that cannot be re-extracted from raw inputs cannot be maintained against a moving threat.

Strengths and weaknesses of the source

- **Strengths:** clear scope, runnable modelling code, a sensible feature family, an honest multi-model comparison, and a correct 'XGBoost is best' conclusion.
- **Weaknesses:** a leaky top feature, non-reproducible WHOIS/Alexa extraction, an unrealistic balanced dataset, accuracy-only reporting, no FP/FN or threshold discussion, and no temporal validation.

Suggestions for future improvement

- Evaluate at realistic prevalence and lead with PR-AUC and MCC.
- Replace Alexa/WHOIS features with stable, reproducible lexical, TLS and passive-DNS features; keep continuous features continuous.
- Use a time-based split to measure and resist concept drift.
- Adopt a richer public corpus with continuous URL features (e.g. the Vrbancic dataset_full.csv, ~88k URLs and 111 numeric features).
- Tune the decision threshold to the false-negative-heavy cost model of phishing.

8. Summing It Up

Problem: automatically classify URLs as phishing or legitimate. **Source:** the shreyagopal 'Phishing Website Detection by Machine Learning Techniques' tutorial. **Dataset:** 10,000 URLs (50/50) with 17 extracted features. **Methodology of our study:** faithful reproduction plus a critical battery: feature-space ambiguity analysis, prevalence correction, a leakage probe, a source-independent model, and a full prevalence-robust metric suite under cross-validation.

Main findings: the author's ~86% accuracy reproduces (86.9% for our best model, Random Forest) but coincides with the 86.8% information ceiling of the binarised features, is inflated by the 50/50 balance (deployment precision ~11.1% at 1% prevalence), and rests partly on a source-revealing feature. The extraction pipeline is not reproducible today. **Were the author's claims supported?** The narrow 'XGBoost performs best' claim, yes; the broad 'effective ~86% phishing detector' claim, no.

Most important insight: on this problem the bottleneck is the feature representation, not the learning algorithm, and the evaluation metric chosen determines whether that bottleneck is even visible. **Recommendation:** valuable as a teaching pipeline, but we would not recommend deploying it, or citing its accuracy, on similar problems without the corrections above. **Final conclusion:** a methodologically instructive project whose central performance claim is not supported by the evidence once rigorous, realistic evaluation is applied.